

main

July 7, 2026

1 Section 1: Data Cleaning

1.1 Import datasets into project and join by shared columns

```
[ ]: # Import OS for accesing system folders
import os as os

# Import Pandas for conversion of .csv into dataframes and usage of Pandas
↳ library
import pandas as pd
import numpy as np

import warnings
warnings.filterwarnings('ignore')
```

```
[ ]: # Set folder_path from our current directory
folder_path = os.getcwd()

# Update folder_path to Data/ folder to access .csv files
folder_path = os.path.join(folder_path, r"Data")
```

```
[ ]: # Check that the files needed are in the folder
os.listdir(folder_path)
```

```
[ ]: ['2012_cleaned.csv', '2013_cleaned.csv', '2014_cleaned.csv']
```

1.1.1 Our first issue with these tables is that not all of them possess the same columns (only the names are the same). Therefore, we need to extract only the columns that all three tables have in common to avoid problems when merging into a bigger dataframe.

```
[ ]: # Proof: columns are different for each .csv

# Loop through each file by name of files as in:
# ['2012_cleaned.csv', '2013_cleaned.csv', '2014_cleaned.csv']
for file_name in os.listdir(folder_path):
    current_file_path = os.path.join(folder_path, file_name)
    current_df = pd.read_csv(
```

```

        current_file_path
    ) # Convert the current .csv into a df to use Pandas columns() module
    print(
        file_name, "has", len(current_df.columns), "columns"
    ) # Check columns are not equal just by looking at amount of columns in
    ↪ each table

```

2012_cleaned.csv has 29 columns

2013_cleaned.csv has 28 columns

2014_cleaned.csv has 37 columns

1.1.2 Our first table has 29, our second 28, and our last one 37. When adding the tables, this will result in columns with many NA values. We must eliminate the “extra” columns first.

```

[ ]: # Run a similar loop as above to created a nested list of all columns, to
    ↪ intersect later

columns_in_tables = []

for file_name in os.listdir(folder_path):
    current_file_path = os.path.join(folder_path, file_name)
    current_df = pd.read_csv(current_file_path)

    # Put all columns for each column in the list created above
    # Will look something like the following:
    # [[table1_column1][table1_column2],[table2_column1]..., [table3_column3]...]
    columns_in_tables.append(current_df.columns)

```

```

[ ]: # For finding which columns are in every table we need to run an intersection
    # Python does this by turning a nested list into a list of sets.

shared_columns = list(
    set.intersection(*map(set, columns_in_tables))
) # * as in *args for multiple arguments input
# The map() function turns each element (sublist of columns) into a set which
# We then turn the set back into a list and store it in shared_columns

```

```

[ ]: # Modify the above for loop to put all tables in a single dataframe and select
    ↪ only shared_columns

data_merge = pd.DataFrame() # Initialize empty df
dataframes = [] # Initialize empty list for use in pd.concat which accepts
    ↪ lists only

for file_name in os.listdir(folder_path):
    current_file_path = os.path.join(folder_path, file_name)

```

```

current_df = pd.read_csv(current_file_path)
dataframes.append(current_df)

data_merge = pd.concat(dataframes)
data_merge = data_merge[list(shared_columns)]

```

1.1.3 We can now see the columns we have to work with.

```

[ ]: data_merge.columns

[ ]: Index(['marriage', 'pre_ele_101', 'wsel', 'zip', 'year', 'religion', 'id',
          'ssu', 'taiwanese_or_chinese', 'edu', 'gender', 'pre_ele_97', 'psu',
          'birth_year', 'mothe_house_income', 'political_party', 'year_m',
          'stratum2', 'wave', 'month_income', 'taiwan_indep', 'qtype'],
          dtype='object')

```

1.1.4 And eliminate those which we don't need

```

[ ]: data_merge = data_merge.drop(
      columns=["year_m", "wave", "psu", "wsel", "id", "qtype", "ssu"]
    )

```

1.2 Normalizing of variables in each column for easier navigation.

1.2.1 We now need to see which values columns have and normalize those values into simpler categories. For example: The political_party columns has the following values which, for this analysis, are too many.

```

[ ]: data_merge["political_party"].value_counts()

```

```

[ ]: political_party
---      1165
      1115
/       838
      749
      605
      406
( h7)   249
( h7)   203
      131
      114
      95
      86
( h7)   41
,       21
      13
( h7)   12
,       12

```

```

12
9
,      8
7
( h7)  3
( h7)  2
1
1
1

```

Name: count, dtype: int64

1.2.2 Or the birth_year column has non-numeric values

```
[ ]: data_merge.dtypes
# Int and floats are numeric while object are usually a combination of strings,
↳ ints, etc...
```

```
[ ]: marriage      object
pre_ele_101      object
zip              object
year             int64
religion         object
taiwanese_or_chinese object
edu              object
gender           object
pre_ele_97       object
birth_year       object
mothe_house_income object
political_party  object
stratum2         object
month_income     object
taiwan_indep     object
dtype: object
```

```
[ ]: # Let's extract columns where birth_year is only numbers...
# Run to_numeric to try and convert to number, if not possible then return NA,
↳ then use notnull to return non-NA values
# i.e. Return only numbers
data_merge = data_merge[
    pd.to_numeric(data_merge["birth_year"], errors="coerce").notnull()
]
data_merge["birth_year"] = data_merge["birth_year"].astype("int")
```

1.2.3 Most of the columns will only require some renaming of variables into categories (e.g. “M” and “Male” should only be one category “Male”). A simple example of renaming can be seen as follows.

```
[ ]: data_merge.loc[data_merge["gender"] == " ", "gender"] = (
    "Male" # Find the values on the left and assign the value on the right
)
data_merge.loc[data_merge["gender"] == " ", "gender"] = "Female"

# Create categories list in order desired (Order can be ignored later.)

gender_categories = [" ", " "]

data_merge["gender"] = pd.Categorical(
    data_merge["gender"], categories=gender_categories
)

data_merge["gender"].unique()
```

```
[ ]: [NaN, ' ', ' ']
Categories (2, object): [' ', ' ']
```

1.2.4 We start by cleaning the political_party target column.

```
[ ]: # Define the values to be replaced and the replacement string
data_merge.loc[
    data_merge["political_party"].isin([" ( h7)", " ", " "]),
    "political_party",
] = "DPP"
data_merge.loc[
    data_merge["political_party"].isin([" ( h7)", " ", " "]),
    "political_party",
] = "KMT"

# Change all other values to 'other'
data_merge.loc[
    ~data_merge["political_party"].isin(["DPP", "KMT"]), "political_party"
] = "Other"
data_merge["political_party"].unique()
```

```
[ ]: array(['Other', 'DPP', 'KMT'], dtype=object)
```

1.2.5 Do a similar process for our ethnic identification column. From this:...

```
[ ]: # Output all values of the ethnic identification category
data_merge["taiwanese_or_chinese"].unique()

[ ]: array([' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
          ' ', ' ':      , ' ':      , ' ', ' ', ' ',
          ' '], dtype=object)
```

1.2.6 To this:

```
[ ]: data_merge["taiwanese_or_chinese"].value_counts()
```

```
[ ]: taiwanese_or_chinese
      3985
      642
      472
      :    455
      144
      :    72
      27
      25
      ,    23
      19
      ,    14
      8
      7
      3
      1
Name: count, dtype: int64
```

```
[ ]: both_categories = [
    " ",
    " ",
    " : ",
    " : ",
    " ",
]

data_merge.loc[
    data_merge["taiwanese_or_chinese"] == " ", "taiwanese_or_chinese"
] = "Chinese"

data_merge.loc[
    data_merge["taiwanese_or_chinese"] == " ", "taiwanese_or_chinese"
] = "Taiwanese"

data_merge.loc[
```

```

data_merge["taiwanese_or_chinese"].isin(both_categories),
↳ "taiwanese_or_chinese"
] = "Both"

data_merge.loc[
    data_merge["taiwanese_or_chinese"].isin([" ", " ", " ", " ", " "]),
    "taiwanese_or_chinese",
] = "Unclear"

data_merge.loc[
    ~data_merge["taiwanese_or_chinese"].isin(
        ["Taiwanese", "Chinese", "Both", "Unclear"]
    ),
    "taiwanese_or_chinese",
] = "Unclear"

data_merge["taiwanese_or_chinese"].value_counts()

```

```

[ ]: taiwanese_or_chinese
Taiwanese    3985
Both         1641
Chinese       144
Unclear       127
Name: count, dtype: int64

```

1.2.7 For the other columns the process looks quite similar

```

[ ]: data_merge["pre_ele_101"].value_counts()

data_merge.loc[data_merge["pre_ele_101"].str.contains(" ", na=False),
↳ "pre_ele_101" = (
    "Tsai Ying-Wen"
)
data_merge.loc[data_merge["pre_ele_101"].str.contains(" ", na=False),
↳ "pre_ele_101" = (
    "Ma Ying-Jiu"
)
data_merge.loc[
    data_merge["pre_ele_101"].str.contains(" | ", na=False), "pre_ele_101"
] = "Other candidate / blank vote"
data_merge.loc[
    data_merge["pre_ele_101"].str.contains(" | | 20 ", na=False),
    "pre_ele_101",
] = "Non-voter / ineligible"
data_merge.loc[
    data_merge["pre_ele_101"].str.contains(" | | | ", na=False),
    "pre_ele_101",
] = "Non-voter / ineligible"

```

```

] = "Unclear"
data_merge.loc[
    ~data_merge["pre_ele_101"].isin(
        [
            "Tsai Ying-Wen",
            "Ma Ying-Jiu",
            "Other candidate / blank vote",
            "Non-voter / ineligible",
            "Unclear",
        ]
    ),
    "pre_ele_101",
] = "Unclear"
data_merge["pre_ele_101"].value_counts()

```

```

[ ]: pre_ele_101
Ma Ying-Jiu                2264
Tsai Ying-Wen              1571
Non-voter / ineligible     1292
Unclear                    558
Other candidate / blank vote 212
Name: count, dtype: int64

```

```

[ ]: data_merge.loc[data_merge["pre_ele_97"].str.contains(" ", na=False),
    ↪ "pre_ele_97"] = (
        "Frank Hsieh"
    )
data_merge.loc[data_merge["pre_ele_97"].str.contains(" ", na=False),
    ↪ "pre_ele_97"] = (
        "Ma Ying-Jiu"
    )
data_merge.loc[
    data_merge["pre_ele_97"].str.contains(" ", na=False), "pre_ele_97"
] = "Other candidate / blank vote"
data_merge.loc[
    data_merge["pre_ele_97"].str.contains(
        " | | 20 | 20 | 20 | ",
        na=False,
    ),
    "pre_ele_97",
] = "Non-voter / ineligible"
data_merge.loc[
    data_merge["pre_ele_97"].str.contains(" | | | ", na=False),
    "pre_ele_97",
] = "Unclear"
data_merge.loc[
    ~data_merge["pre_ele_97"].isin(

```



```

        [
            "Frank Hsieh",
            "Ma Ying-Jiu",
            "Other candidate / blank vote",
            "Non-voter / ineligible",
            "Unclear",
        ]
    ),
    "pre_ele_97",
] = "Unclear"

data_merge["pre_ele_97"].value_counts()

```

```

[ ]: pre_ele_97
Non-voter / ineligible      2125
Ma Ying-Jiu                1672
Unclear                    1248
Frank Hsieh                 785
Other candidate / blank vote  67
Name: count, dtype: int64

```

```

[ ]: data_merge.loc[data_merge["edu"].str.contains("/ | / / ", na=False), "edu"] =
    ↪(
        "No Formal Education"
    )
data_merge.loc[data_merge["edu"].str.contains(" ", na=False), "edu"] = "Primary_
    ↪Education"
data_merge.loc[
    data_merge["edu"].str.contains("\\(\\) | | | | ", na=False),
    "edu",
] = "Secondary Education"
data_merge.loc[
    data_merge["edu"].str.contains(" | | | | ", na=False),
    "edu",
] = "Military Education"
data_merge.loc[
    data_merge["edu"].str.contains(" | | | / ", na=False), "edu"
] = "Associate Education"
data_merge.loc[
    data_merge["edu"].str.contains(" | | | ", na=False), "edu"
] = "Undergraduate"
data_merge.loc[data_merge["edu"].str.contains(" ", na=False), "edu"] =
    ↪"Graduate"
data_merge.loc[data_merge["edu"].str.contains(" ", na=False), "edu"] =
    ↪"Doctorate"

# Set the non-above values to an explicit uncertainty bucket

```

```

data_merge.loc[
    ~data_merge["edu"].isin(
        [
            "No Formal Education",
            "Primary Education",
            "Secondary Education",
            "Military Education",
            "Associate Education",
            "Undergraduate",
            "Graduate",
            "Doctorate",
        ]
    ),
    "edu",
] = "Unclear"

# Create sorted value list for the education category
education_categories = [
    "Unclear",
    "No Formal Education",
    "Primary Education",
    "Secondary Education",
    "Military Education",
    "Associate Education",
    "Undergraduate",
    "Graduate",
    "Doctorate",
]

# Transform education column into categorical for sorting by the above list
data_merge["edu"] = pd.Categorical(data_merge["edu"],  

    ↪categories=education_categories)

data_merge["edu"].value_counts()

```

```

[ ]: edu
Secondary Education    2207
Undergraduate          1441
Primary Education      830
Associate Education    648
Graduate               351
No Formal Education    306
Military Education     79
Doctorate              30
Unclear                5
Name: count, dtype: int64

```

```
[ ]: data_merge.loc[data_merge["marriage"].str.contains("      | ", na=False),
    ↪ "marriage"] = (
        "Married (Live Together)"
    )
data_merge.loc[
    data_merge["marriage"].str.contains("      |      | ", na=False),
    "marriage",
] = "Married (Live Separately)"
data_merge.loc[data_merge["marriage"].str.contains(" ", na=False), "marriage"]
    ↪ = "Cohabiting"
data_merge.loc[data_merge["marriage"].str.contains(" ", na=False), "marriage"]
    ↪ = "Divorced"
data_merge.loc[data_merge["marriage"].str.contains(" ", na=False), "marriage"]
    ↪ = "Widowed"
data_merge.loc[data_merge["marriage"].str.contains("      ", na=False),
    ↪ "marriage"] = "Single"
data_merge.loc[data_merge["marriage"].str.contains(" | ", na=False),
    ↪ "marriage"] = "Unclear"

data_merge.loc[
    ~data_merge["marriage"].isin(
        [
            "Married (Live Together)",
            "Married (Live Separately)",
            "Cohabiting",
            "Divorced",
            "Widowed",
            "Single",
            "Unclear",
        ]
    ),
    "marriage",
] = "Unclear"

data_merge["marriage"].value_counts()
```

```
[ ]: marriage
Married (Live Together)    3403
Single                    1679
Widowed                   445
Divorced                  247
Married (Live Separately)   84
Cohabiting                 37
Unclear                    2
Name: count, dtype: int64
```

```

[ ]: # Put unclear variables into variables
data_merge.loc[
    data_merge["mothe_house_income"].str.contains(" | ", na=False),
    "mothe_house_income",
] = "Unclear"

# Clean "repeated" variables
data_merge.loc[data_merge["mothe_house_income"] == " ", "mothe_house_income"] =
    "No Income"

# Create sorted value list for the salary category
salary_categories = [
    "No Income",
    "1 ",
    "1-2 ",
    "2-3 ",
    "3-4 ",
    "4-5 ",
    "5-6 ",
    "6-7 ",
    "7-8 ",
    "8-9 ",
    "9-10 ",
    "10-11 ",
    "11-12 ",
    "12-13 ",
    "13-14 ",
    "14-15 ",
    "15-16 ",
    "16-17 ",
    "17-18 ",
    "18-19 ",
    "19-20 ",
    "20-30 ",
    "30-40 ",
    "40-50 ",
    "50-100 ",
    "100 ",
    "Unclear",
]

# Transform income column into categorical for sorting by the above list
data_merge["mothe_house_income"] = pd.Categorical(
    data_merge["mothe_house_income"], categories=salary_categories, ordered=True
)

```

```

[ ]: # Put unclear variables into variables
data_merge.loc[
    data_merge["month_income"].str.contains(" | | ", na=False),
    "month_income",
] = "Unclear"

# Clean "repeated" variables
data_merge.loc[
    data_merge["month_income"].str.contains(" | | , ", na=False),
    "month_income",
] = "No Income"

# Create sorted value list for the salary category
salary_categories = [
    "No Income",
    "1 ",
    "1-2 ",
    "2-3 ",
    "3-4 ",
    "4-5 ",
    "5-6 ",
    "6-7 ",
    "7-8 ",
    "8-9 ",
    "9-10 ",
    "10-11 ",
    "11-12 ",
    "12-13 ",
    "13-14 ",
    "14-15 ",
    "15-16 ",
    "16-17 ",
    "17-18 ",
    "18-19 ",
    "19-20 ",
    "20-30 ",
    "30-40 ",
    "40-50 ",
    "50-100 ",
    "100 ",
    "30 ",
    "Unclear",
]

# Transform income column into categorical for sorting by the above list
data_merge["month_income"] = pd.Categorical(
    data_merge["month_income"], categories=salary_categories, ordered=True

```

```
)
```

```
[ ]: data_merge.loc[data_merge["taiwan_indep"].str.contains("    | ", na=False),  
    ↪ "taiwan_indep"] = (  
        "Status Quo"  
    )  
  
data_merge.loc[data_merge["taiwan_indep"].str.contains(" .* ", na=False),  
    ↪ "taiwan_indep"] = (  
        "Independence"  
    )  
  
data_merge.loc[  
    data_merge["taiwan_indep"].str.contains(" .* | ", na=False), "taiwan_indep"  
] = "Unification"  
  
data_merge.loc[  
    data_merge["taiwan_indep"].str.contains(" | | | | ", na=False),  
    "taiwan_indep",  
] = "Unclear"  
  
data_merge.loc[  
    ~data_merge["taiwan_indep"].isin(["Status Quo", "Independence",  
    ↪ "Unification", "Unclear"]),  
    "taiwan_indep",  
] = "Unclear"  
  
data_merge["taiwan_indep"].value_counts()
```

```
[ ]: taiwan_indep  
Status Quo      4967  
Independence    450  
Unclear         363  
Unification     117  
Name: count, dtype: int64
```

```
[ ]: data_merge['religion'].value_counts()
```

```
[ ]: religion  
1565  
780  
560  
379  
, 264  
176  
, 107
```

```

        66
        36
        7
    ,
        4
        2
        1
        1
Name: count, dtype: int64

```

```
[ ]: data_merge.columns
```

```
[ ]: Index(['marriage', 'pre_ele_101', 'zip', 'year', 'religion',
          'taiwanese_or_chinese', 'edu', 'gender', 'pre_ele_97', 'birth_year',
          'mothe_house_income', 'political_party', 'stratum2', 'month_income',
          'taiwan_indep'],
          dtype='object')
```

```
[ ]: data_merge.loc[data_merge["religion"].isna(), "religion"] = "Unclear"

data_merge.loc[data_merge["religion"].str.contains(" ", na=False),
               ↪ "religion"] = "No Religion"
data_merge.loc[data_merge["religion"].str.contains(" ", na=False), "religion"]
               ↪= "Folk Religion"
data_merge.loc[data_merge["religion"].str.contains(" | ", na=False),
               ↪ "religion"] = "Buddhism"
data_merge.loc[data_merge["religion"].str.contains(" ", na=False), "religion"]
               ↪= "Christianity"
data_merge.loc[data_merge["religion"].str.contains(" ", na=False), "religion"]
               ↪= "Catholicism"
data_merge.loc[data_merge["religion"].str.contains(" ", na=False), "religion"]
               ↪= "Taoism"
data_merge.loc[data_merge["religion"].str.contains(" ", na=False), "religion"]
               ↪= "Yiguandao"
data_merge.loc[data_merge["religion"].str.contains(" ", na=False), "religion"]
               ↪= "Confucianism"
data_merge.loc[
    data_merge["religion"].str.contains(" | | ", na=False),
    "religion",
] = "Unclear"

data_merge.loc[
    ~data_merge["religion"].isin(
        [
            "No Religion",
            "Folk Religion",
            "Buddhism",
            "Christianity",

```

```

        "Catholicism",
        "Taoism",
        "Yiguandao",
        "Confucianism",
        "Unclear",
    ]
),
    "religion",
] = "Unclear"

data_merge["religion"].value_counts()

```

```

[ ]: religion
Unclear          1961
Folk Religion    1565
No Religion       780
Buddhism         751
Taoism           560
Christianity     176
Yiguandao        66
Catholicism      36
Confucianism      2
Name: count, dtype: int64

```

1.3 Convert non-categorical columns to categorical

```

[ ]: # Finish turning remaining columns into categorical columns based on their
    ↪ post-cleaning unique values
for column in data_merge:
    if data_merge[column].dtype in ["object", "string"]:
        unique_to_cat = data_merge[column].unique()
        data_merge[column] = pd.Categorical(
            data_merge[column], categories=unique_to_cat
        )

```

1.3.1 Compare with column type at the beginning of cleaning section...

```

[ ]: data_merge.dtypes

```

```

[ ]: marriage          category
pre_ele_101          category
zip                  category
year                 int64
religion             category
taiwanese_or_chinese category
edu                  category
gender              category

```



```
pre_ele_97          category
birth_year          int64
mothe_house_income  category
political_party     category
stratum2           category
month_income        category
taiwan_indep        category
dtype: object
```

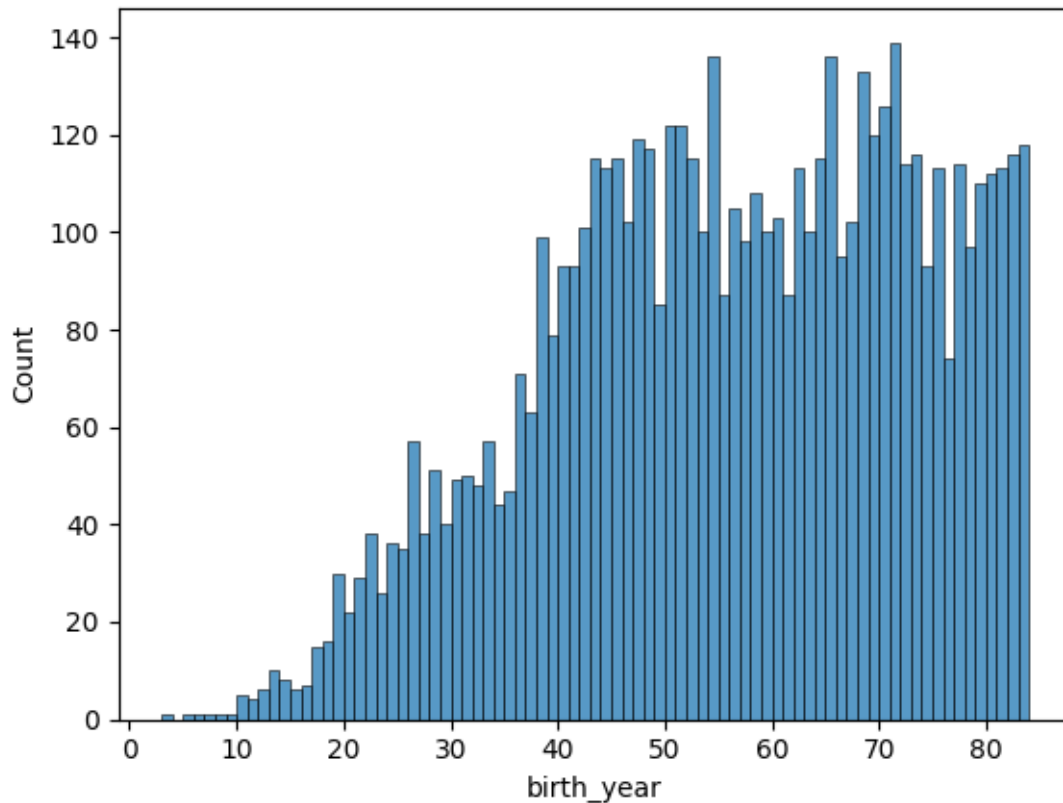
2 Section 2: Visualization

```
[ ]: #Import our visualization tool Plotnine
import matplotlib.pyplot as plt
import seaborn as sns

# sns.set(style="whitegrid")
sns.set_color_codes("pastel")
# sns.set(font="SimSun")
%matplotlib inline
```

2.1 A quick visualization:

```
[ ]: sns.histplot(data_merge["birth_year"], binwidth=1)
plt.show()
```



2.1.1 We can see as follows the distribution of political parties by self-identification and vice-versa, self-identification by political party.

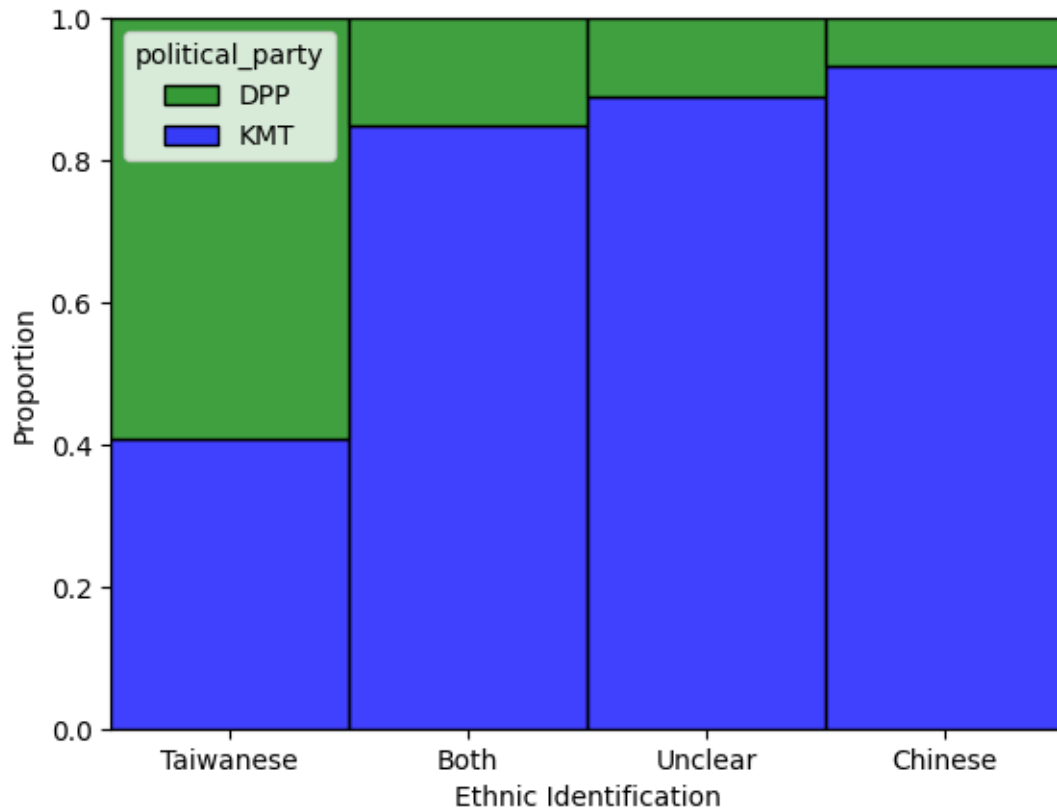
```
[ ]: df_parties = data_merge[data_merge["political_party"].isin(["DPP", "KMT"])]
```

```
[ ]: color_palette_parties = {"DPP": "green", "KMT": "blue"}
```

```
ax = sns.histplot(
    data=data_merge,
    x="taiwanese_or_chinese",
    hue="political_party",
    hue_order=["DPP", "KMT"],
    multiple="fill",
    stat="proportion",
    palette=color_palette_parties,
)

ax.set(xlabel="Ethnic Identification")
```

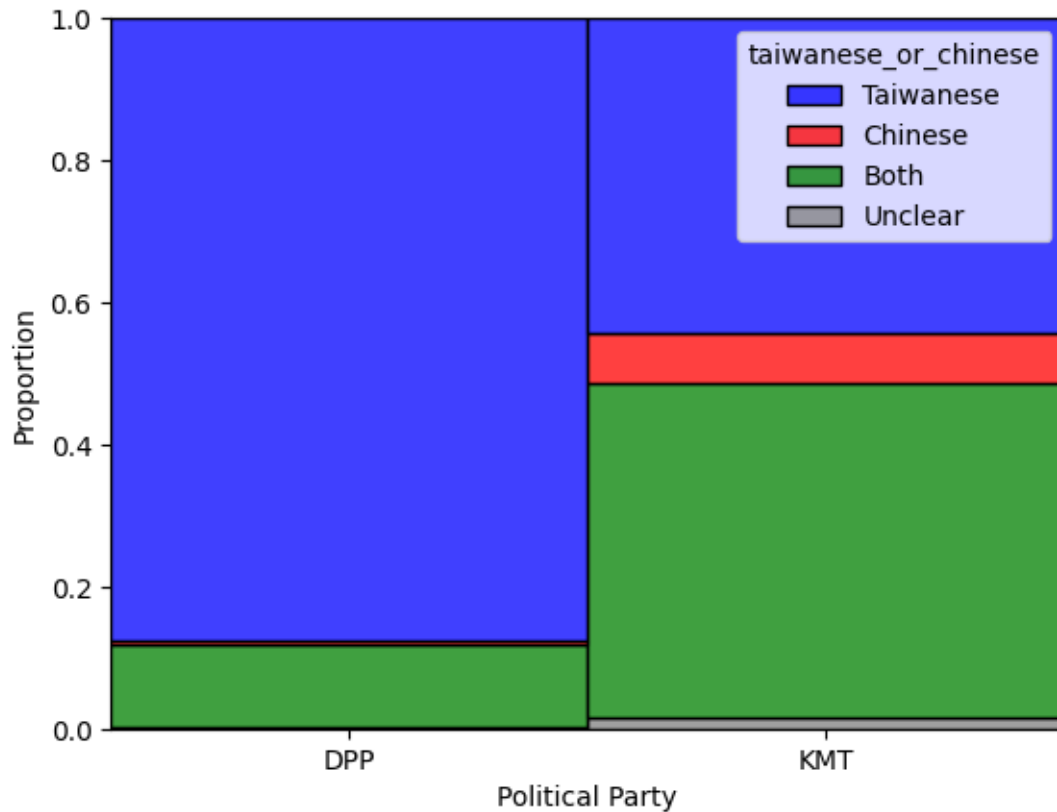
```
[ ]: [Text(0.5, 0, 'Ethnic Identification')]
```



```
[ ]: color_palette_ethnic = {
    "Taiwanese": "blue",
    "Chinese": "red",
    "Both": "green",
    "Unclear": "gray",
}

ax = sns.histplot(
    data=df_parties,
    x="political_party",
    hue="taiwanese_or_chinese",
    hue_order=["Taiwanese", "Chinese", "Both", "Unclear"],
    multiple="fill",
    stat="proportion",
    palette=color_palette_ethnic,
)
ax.set(xlabel="Political Party")
```

```
[ ]: [Text(0.5, 0, 'Political Party')]
```



People that identify as Chinese are more probable to vote KMT than other self-identifications.

Although the margin is small, KMT voters tend to have a significantly higher amount of people that self-identify as Chinese.

3 Step 3: Machine Learning Classification

```
[ ]: from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OrdinalEncoder
from sklearn.metrics import accuracy_score, recall_score, f1_score, precision_score
```

3.0.1 Before we begin building our models, we must eliminate unimportant values from our target column, such a `political_party == 'Other'`. This is because 'Other' is a majority class and could imbalance our classification model leading to subpar metrics.

3.0.2 'Other' is almost four times more sampled than the next biggest category, this majority representation can introduce bias in our model.

```
[ ]: data_merge["political_party"].value_counts()
```

```
[ ]: political_party
      Other    4190
      KMT      985
      DPP      722
      Name: count, dtype: int64
```

3.0.3 The following model shows metrics for prediction when 'Other' is not removed. Observe that party classification models' non-Accuracy metrics are worse than the 'Other' category

```
[ ]: # Split the dataset into features (X) and target variable (y)
      X = data_merge.drop(columns=["political_party"]) # Drop non-target columns
      y = data_merge["political_party"] # Create target variable

      # Our models require us to encode the categories in our feature set
      X_encoded = pd.get_dummies(X, columns=X.select_dtypes(include=["category"]).
      ↪columns)
```

```
[ ]: # Split the dataset into training and testing sets
      X_train, X_test, y_train, y_test = train_test_split(
          X_encoded, y, test_size=0.2, random_state=42
      )

      # Initialize the Random Forest classifier
      rf_classifier = (
          RandomForestClassifier(class_weight="balanced", random_state=42)
      ) # You can adjust the number of trees (n_estimators) and other hyperparameters

      # Train the classifier on the training data
      rf_classifier.fit(X_train, y_train)

      # Make predictions on the test data
      y_pred = rf_classifier.predict(X_test)

      # Evaluate the accuracy of the classifier
      accuracy = accuracy_score(y_test, y_pred)
      recall = recall_score(y_test, y_pred, average="weighted")
```

```

f1 = f1_score(y_test, y_pred, average="weighted")
precision = precision_score(y_test, y_pred, average="weighted")
print(
    "For the Random Forest predictor model:",
    "\n Accuracy",
    accuracy,
    "\n Recall is:",
    recall,
    "\n F1 Score:",
    f1,
    "\n Precision",
    precision,
)

feat_importances = pd.Series(
    rf_classifier.feature_importances_, index=X_encoded.columns
)
ax = feat_importances.nlargest(20).plot(kind="barh")
ax.invert_yaxis() # Invert y-axis
plt.title("Top 20 Feature Importances")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.show() # Display the plot

```

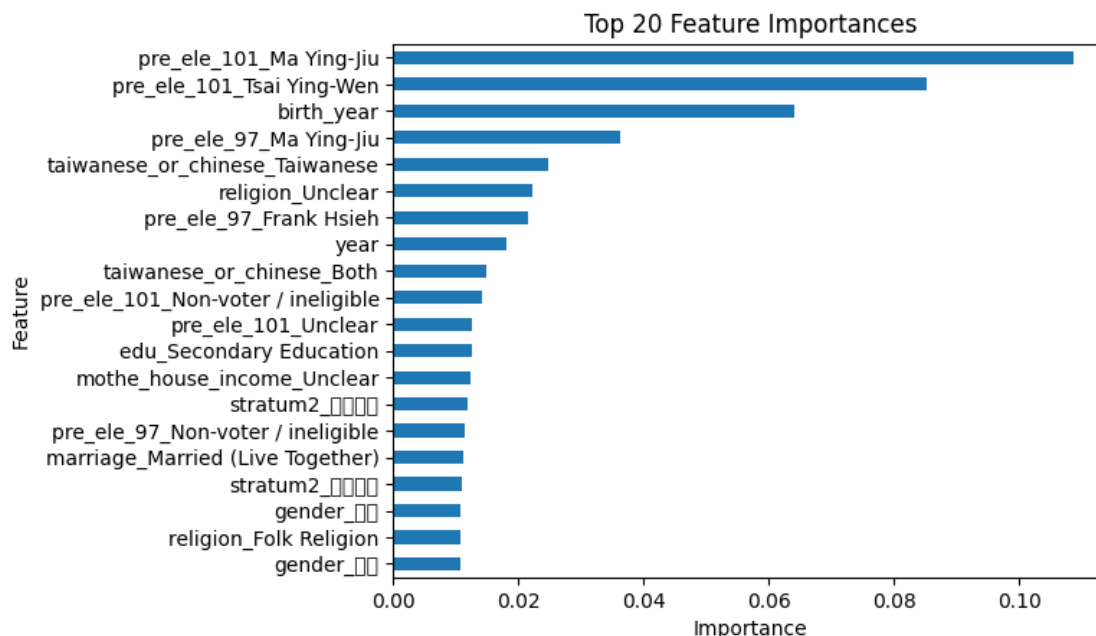
For the Random Forest predictor model:

Accuracy 0.7584745762711864

Recall is: 0.7584745762711864

F1 Score: 0.7440251327485741

Precision 0.7448811876197732



3.0.4 For the remaining of our modelling, we eliminate this ‘Other’ category.

```
[ ]: data_merge = data_merge[data_merge["political_party"] != "Other"] # Eliminate
    ↳ the 'other' category
```

```
[ ]: # Split the dataset into features (X) and target variable (y)
X = data_merge.drop(columns=["political_party"]) # Drop non-target columns
y = data_merge["political_party"] # Create target variable

# Our models require us to encode the categories in our feature set
X_encoded = pd.get_dummies(X, columns=X.select_dtypes(include=["category"]).
    ↳ columns)
```

```
[ ]: data_merge['political_party'].value_counts()
```

```
[ ]: political_party
KMT      985
DPP      722
Other      0
Name: count, dtype: int64
```

```
[ ]: # Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.2, random_state=42
)

# Initialize the Random Forest classifier
rf_classifier = (
    RandomForestClassifier(class_weight="balanced", random_state=42)
) # You can adjust the number of trees (n_estimators) and other hyperparameters

# Train the classifier on the training data
rf_classifier.fit(X_train, y_train)

# Make predictions on the test data
y_pred = rf_classifier.predict(X_test)

# Evaluate the accuracy of the classifier
accuracy = accuracy_score(y_test, y_pred)
recall = recall_score(y_test, y_pred, average="weighted")
f1 = f1_score(y_test, y_pred, average="weighted")
precision = precision_score(y_test, y_pred, average="weighted")
print(
    "For the Random Forest predictor model:",
```

```

    "\n Accuracy",
    accuracy,
    "\n Recall is:",
    recall,
    "\n F1 Score:",
    f1,
    "\n Precision",
    precision,
)

feat_importances = pd.Series(
    rf_classifier.feature_importances_, index=X_encoded.columns
)
ax = feat_importances.nlargest(20).plot(kind="barh")
ax.invert_yaxis() # Invert y-axis
plt.title("Top 20 Feature Importances")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.show() # Display the plot

```

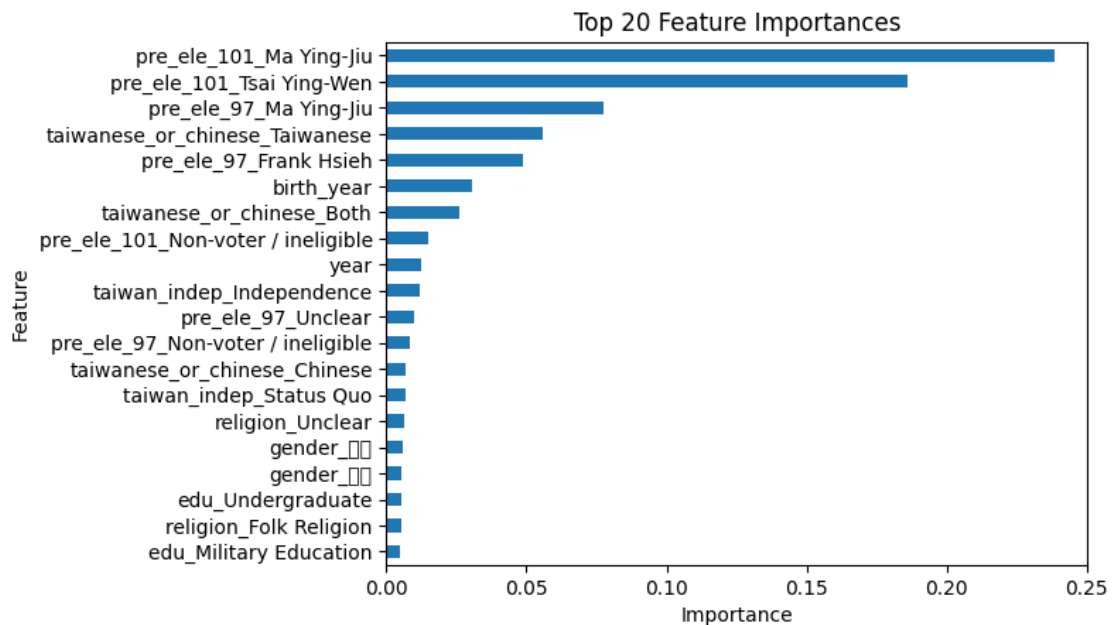
For the Random Forest predictor model:

Accuracy 0.9269005847953217

Recall is: 0.9269005847953217

F1 Score: 0.9264832395998231

Precision 0.9274137431660867




```
[ ]: from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred, target_names=["KMT", "DPP"]))
```

	precision	recall	f1-score	support
KMT	0.94	0.88	0.91	140
DPP	0.92	0.96	0.94	202
accuracy			0.93	342
macro avg	0.93	0.92	0.92	342
weighted avg	0.93	0.93	0.93	342

Recall, precision and F1 Scores are much improved thanks to target feature simplification

3.0.5 The cells below build a train/test split once and then compare a baseline tree, a pruned tree selected with 5-fold cross validation, a tuned random forest, and a tuned boosted tree.

```
[ ]: from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold,
    ↪cross_val_score, train_test_split
from sklearn.metrics import accuracy_score, f1_score, precision_score,
    ↪recall_score

# Keep the binary target used in the existing modeling section
model_df = data_merge[data_merge["political_party"].isin(["DPP", "KMT"])].copy()
X_model = model_df.drop(columns=["political_party"])
y_model = model_df["political_party"]

# One-hot encode the cleaned categorical predictors
X_model_encoded = pd.get_dummies(
    X_model, columns=X_model.select_dtypes(include=["category", "object"]).
    ↪columns
)

# Construct a single training set and test set for all model comparisons
X_train, X_test, y_train, y_test = train_test_split(
    X_model_encoded,
    y_model,
    test_size=0.2,
    random_state=42,
    stratify=y_model,
)

cv5 = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```

print("Training rows:", len(X_train))
print("Test rows:", len(X_test))
print("Encoded feature count:", X_train.shape[1])

```

Training rows: 1365
 Test rows: 342
 Encoded feature count: 232

```

[ ]: # Baseline tree without pruning
baseline_tree = DecisionTreeClassifier(random_state=42)
baseline_tree.fit(X_train, y_train)
baseline_pred = baseline_tree.predict(X_test)

print("Baseline decision tree")
print("Accuracy:", accuracy_score(y_test, baseline_pred))
print("Recall:", recall_score(y_test, baseline_pred, average="weighted",
    ↳zero_division=0))
print("F1 score:", f1_score(y_test, baseline_pred, average="weighted",
    ↳zero_division=0))
print("Precision:", precision_score(y_test, baseline_pred, average="weighted",
    ↳zero_division=0))
print("Tree depth:", baseline_tree.get_depth())
print("Tree leaves:", baseline_tree.get_n_leaves())
print("Tree splits:", baseline_tree.tree_.node_count - baseline_tree.
    ↳get_n_leaves())

```

Baseline decision tree
 Accuracy: 0.9152046783625731
 Recall: 0.9152046783625731
 F1 score: 0.9153134823593321
 Precision: 0.9155651853698518
 Tree depth: 17
 Tree leaves: 100
 Tree splits: 99

```

[ ]: # Cross-validated cost-complexity pruning
pruning_path_tree = DecisionTreeClassifier(random_state=42)
pruning_path = pruning_path_tree.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = np.unique(pruning_path.ccp_alphas)

# Drop the largest alpha because it usually collapses the tree to a single node
if len(ccp_alphas) > 1:
    ccp_alphas = ccp_alphas[:-1]

pruning_results = []
for alpha in ccp_alphas:

```

```

    candidate_tree = DecisionTreeClassifier(random_state=42,
    ↪ccp_alpha=float(alpha))
    cv_scores = cross_val_score(
        candidate_tree,
        X_train,
        y_train,
        cv=cv5,
        scoring="accuracy",
        n_jobs=-1,
    )
    pruning_results.append(
        {
            "ccp_alpha": float(alpha),
            "mean_accuracy": cv_scores.mean(),
            "std_accuracy": cv_scores.std(),
        }
    )

pruning_results = pd.DataFrame(pruning_results)
best_row = pruning_results.loc[pruning_results["mean_accuracy"].idxmax()]

# One-standard-error rule: keep the simplest tree within one std of the best
↪mean score
candidate_rows = pruning_results[
    pruning_results["mean_accuracy"] >= best_row["mean_accuracy"] -
    ↪best_row["std_accuracy"]
]
chosen_row = candidate_rows.sort_values("ccp_alpha", ascending=False).iloc[0]
chosen_alpha = float(chosen_row["ccp_alpha"])

pruned_tree = DecisionTreeClassifier(random_state=42, ccp_alpha=chosen_alpha)
pruned_tree.fit(X_train, y_train)
pruned_pred = pruned_tree.predict(X_test)

print("Pruned decision tree")
print("Chosen ccp_alpha:", chosen_alpha)
print("Cross-val mean accuracy:", chosen_row["mean_accuracy"])
print("Cross-val std accuracy:", chosen_row["std_accuracy"])
print("Accuracy:", accuracy_score(y_test, pruned_pred))
print("Recall:", recall_score(y_test, pruned_pred, average="weighted",
    ↪zero_division=0))
print("F1 score:", f1_score(y_test, pruned_pred, average="weighted",
    ↪zero_division=0))
print("Precision:", precision_score(y_test, pruned_pred, average="weighted",
    ↪zero_division=0))
print("Tree depth:", pruned_tree.get_depth())

```

```

print("Tree leaves:", pruned_tree.get_n_leaves())
print("Tree splits:", pruned_tree.tree_.node_count - pruned_tree.get_n_leaves())

print("Top pruning candidates by accuracy:")
print(pruning_results.sort_values(["mean_accuracy", "ccp_alpha"],
    ↪ascending=[False, False]).head(10))

```

Pruned decision tree

Chosen ccp_alpha: 0.004447829126986602

Cross-val mean accuracy: 0.9113553113553113

Cross-val std accuracy: 0.02332847374079216

Accuracy: 0.8918128654970761

Recall: 0.8918128654970761

F1 score: 0.8924342826040528

Precision: 0.9011165365521083

Tree depth: 3

Tree leaves: 5

Tree splits: 4

Top pruning candidates by accuracy:

	ccp_alpha	mean_accuracy	std_accuracy
22	0.001583	0.921612	0.013227
28	0.003423	0.919414	0.017643
24	0.001693	0.919414	0.013308
23	0.001686	0.919414	0.013308
20	0.001251	0.918681	0.016283
27	0.002718	0.918681	0.018446
26	0.002016	0.918681	0.018735
25	0.001963	0.918681	0.018735
21	0.001364	0.917949	0.015983
18	0.001210	0.917216	0.015297

```

[ ]: # Random forest tuned with RandomizedSearchCV
rf_base = RandomForestClassifier(class_weight="balanced", random_state=42)
rf_param_distributions = {
    "n_estimators": [200, 300, 500, 700],
    "max_depth": [None, 10, 20, 30, 40],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "max_features": [None, "sqrt", "log2"],
    "bootstrap": [True, False],
}

rf_search = RandomizedSearchCV(
    estimator=rf_base,
    param_distributions=rf_param_distributions,
    n_iter=20,
    cv=cv5,

```

```

        scoring="f1_weighted",
        random_state=42,
        n_jobs=-1,
        verbose=1,
    )
    rf_search.fit(X_train, y_train)
    rf_best = rf_search.best_estimator_
    rf_pred = rf_best.predict(X_test)

    print("Random forest with RandomizedSearchCV")
    print("Best parameters:", rf_search.best_params_)
    print("Best CV score:", rf_search.best_score_)
    print("Accuracy:", accuracy_score(y_test, rf_pred))
    print("Recall:", recall_score(y_test, rf_pred, average="weighted",
        ↪zero_division=0))
    print("F1 score:", f1_score(y_test, rf_pred, average="weighted",
        ↪zero_division=0))
    print("Precision:", precision_score(y_test, rf_pred, average="weighted",
        ↪zero_division=0))
    print("Bagging-style option was available when max_features=None")

```

Fitting 5 folds for each of 20 candidates, totalling 100 fits
 Random forest with RandomizedSearchCV
 Best parameters: {'n_estimators': 500, 'min_samples_split': 2,
 'min_samples_leaf': 2, 'max_features': 'sqrt', 'max_depth': 10, 'bootstrap':
 True}
 Best CV score: 0.9335361386726324
 Accuracy: 0.9298245614035088
 Recall: 0.9298245614035088
 F1 score: 0.9300804093567251
 Precision: 0.9319369667916557
 Bagging-style option was available when max_features=None

```

[ ]: # Boosted tree tuned with cross validation
boosted_tree = GradientBoostingClassifier(random_state=42)
boosted_param_distributions = {
    "n_estimators": [100, 150, 200, 300],
    "learning_rate": [0.01, 0.05, 0.1, 0.2],
    "max_depth": [1, 2, 3, 4],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "subsample": [0.6, 0.8, 1.0],
    "max_features": [None, "sqrt", "log2"],
}

boosted_search = RandomizedSearchCV(
    estimator=boosted_tree,

```

```

    param_distributions=boosted_param_distributions,
    n_iter=20,
    cv=cv5,
    scoring="f1_weighted",
    random_state=42,
    n_jobs=-1,
    verbose=1,
)
boosted_search.fit(X_train, y_train)
boosted_best = boosted_search.best_estimator_
boosted_pred = boosted_best.predict(X_test)

print("Boosted tree with RandomizedSearchCV")
print("Best parameters:", boosted_search.best_params_)
print("Best CV score:", boosted_search.best_score_)
print("Accuracy:", accuracy_score(y_test, boosted_pred))
print("Recall:", recall_score(y_test, boosted_pred, average="weighted",
    ↪zero_division=0))
print("F1 score:", f1_score(y_test, boosted_pred, average="weighted",
    ↪zero_division=0))
print("Precision:", precision_score(y_test, boosted_pred, average="weighted",
    ↪zero_division=0))

```

Fitting 5 folds for each of 20 candidates, totalling 100 fits

Boosted tree with RandomizedSearchCV

Best parameters: {'subsample': 0.8, 'n_estimators': 300, 'min_samples_split': 10, 'min_samples_leaf': 1, 'max_features': 'sqrt', 'max_depth': 2, 'learning_rate': 0.01}

Best CV score: 0.9265590305911893

Accuracy: 0.9298245614035088

Recall: 0.9298245614035088

F1 score: 0.9299932053594562

Precision: 0.9307406549034423

```
[ ]: !pip install xgboost catboost
```

Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.3.0)

Collecting catboost

Downloading catboost-1.2.10-cp312-cp312-manylinux2014_x86_64.whl.metadata (1.2 kB)

Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.0.2)

Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.30.7)

Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from xgboost) (1.16.3)

Requirement already satisfied: graphviz in /usr/local/lib/python3.12/dist-

```

packages (from catboost) (0.21)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-
packages (from catboost) (3.10.0)
Requirement already satisfied: pandas<4.0,>=0.24 in
/usr/local/lib/python3.12/dist-packages (from catboost) (2.2.2)
Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packages
(from catboost) (5.24.1)
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages
(from catboost) (1.17.0)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.12/dist-packages (from pandas<4.0,>=0.24->catboost)
(2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-
packages (from pandas<4.0,>=0.24->catboost) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-
packages (from pandas<4.0,>=0.24->catboost) (2026.2)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (1.3.3)
Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.12/dist-
packages (from matplotlib->catboost) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (1.5.0)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (26.2)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-
packages (from matplotlib->catboost) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->catboost) (3.3.2)
Requirement already satisfied: tenacity>=6.2.0 in
/usr/local/lib/python3.12/dist-packages (from plotly->catboost) (9.1.4)
Downloading catboost-1.2.10-cp312-cp312-manylinux2014_x86_64.whl (97.1 MB)
      97.1/97.1 MB
6.0 MB/s eta 0:00:00
Installing collected packages: catboost
Successfully installed catboost-1.2.10

```

```

[ ]: from xgboost import XGBClassifier
     from catboost import CatBoostClassifier
     from sklearn.preprocessing import LabelEncoder

```

```

[ ]: # XGBoost requires numeric labels
     le = LabelEncoder()
     y_train_enc = le.fit_transform(y_train)
     y_test_enc = le.transform(y_test)

```

```

# XGBoost
xgb_base = XGBClassifier(
    eval_metric="logloss",
    random_state=42,
    n_jobs=-1,
)

xgb_param_distributions = {
    "n_estimators": [100, 200, 300, 500],
    "learning_rate": [0.01, 0.05, 0.1, 0.2],
    "max_depth": [3, 4, 5, 6],
    "subsample": [0.6, 0.8, 1.0],
    "colsample_bytree": [0.6, 0.8, 1.0],
    "min_child_weight": [1, 3, 5],
    "gamma": [0, 0.1, 0.3],
}

xgb_search = RandomizedSearchCV(
    estimator=xgb_base,
    param_distributions=xgb_param_distributions,
    n_iter=20,
    cv=cv5,
    scoring="f1_weighted",
    random_state=42,
    n_jobs=-1,
    verbose=1,
)

xgb_search.fit(X_train, y_train_enc)
xgb_best = xgb_search.best_estimator_
xgb_pred = le.inverse_transform(xgb_best.predict(X_test))

print("XGBoost with RandomizedSearchCV")
print("Best parameters:", xgb_search.best_params_)
print("Best CV score:", xgb_search.best_score_)
print("Accuracy:", accuracy_score(y_test, xgb_pred))
print("Recall:", recall_score(y_test, xgb_pred, average="weighted",
    ↪zero_division=0))
print("F1 score:", f1_score(y_test, xgb_pred, average="weighted",
    ↪zero_division=0))
print("Precision:", precision_score(y_test, xgb_pred, average="weighted",
    ↪zero_division=0))

```

Fitting 5 folds for each of 20 candidates, totalling 100 fits

XGBoost with RandomizedSearchCV

Best parameters: {'subsample': 0.6, 'n_estimators': 200, 'min_child_weight': 1, 'max_depth': 3, 'learning_rate': 0.1, 'gamma': 0.3, 'colsample_bytree': 0.8}

Best CV score: 0.9266429194007776

Accuracy: 0.9473684210526315

Recall: 0.9473684210526315
F1 score: 0.9475294802363908
Precision: 0.948737129679682

```
[ ]: cb_base = CatBoostClassifier(
    random_state=42,
    verbose=0,
)
cb_param_distributions = {
    "iterations": [100, 200, 300, 500],
    "learning_rate": [0.01, 0.05, 0.1, 0.2],
    "depth": [3, 4, 5, 6],
    "l2_leaf_reg": [1, 3, 5, 7],
    "subsample": [0.6, 0.8, 1.0],
    "colsample_bylevel": [0.6, 0.8, 1.0],
}

cb_search = RandomizedSearchCV(
    estimator=cb_base,
    param_distributions=cb_param_distributions,
    n_iter=20,
    cv=cv5,
    scoring="f1_weighted",
    random_state=42,
    n_jobs=-1,
    verbose=1,
)
cb_search.fit(X_train, y_train)
cb_best = cb_search.best_estimator_
cb_pred = cb_best.predict(X_test)

print("CatBoost with RandomizedSearchCV")
print("Best parameters:", cb_search.best_params_)
print("Best CV score:", cb_search.best_score_)
print("Accuracy:", accuracy_score(y_test, cb_pred))
print("Recall:", recall_score(y_test, cb_pred, average="weighted",
    ↪zero_division=0))
print("F1 score:", f1_score(y_test, cb_pred, average="weighted",
    ↪zero_division=0))
print("Precision:", precision_score(y_test, cb_pred, average="weighted",
    ↪zero_division=0))
```

Fitting 5 folds for each of 20 candidates, totalling 100 fits
CatBoost with RandomizedSearchCV
Best parameters: {'subsample': 1.0, 'learning_rate': 0.1, 'l2_leaf_reg': 5, 'iterations': 100, 'depth': 5, 'colsample_bylevel': 0.8}
Best CV score: 0.9281562013073084
Accuracy: 0.9444444444444444

Recall: 0.9444444444444444
F1 score: 0.9445582133918154
Precision: 0.9450810185185186

```
[ ]: summary = pd.DataFrame(  
    {  
        "Model": [  
            "Baseline Tree",  
            "Pruned Tree",  
            "Random Forest",  
            "Gradient Boosting",  
            "XGBoost",  
            "CatBoost",  
        ],  
        "Accuracy": [  
            accuracy_score(y_test, baseline_pred),  
            accuracy_score(y_test, pruned_pred),  
            accuracy_score(y_test, rf_pred),  
            accuracy_score(y_test, boosted_pred),  
            accuracy_score(y_test, xgb_pred),  
            accuracy_score(y_test, cb_pred),  
        ],  
        "F1 (weighted)": [  
            f1_score(y_test, baseline_pred, average="weighted",  
↪zero_division=0),  
            f1_score(y_test, pruned_pred, average="weighted", zero_division=0),  
            f1_score(y_test, rf_pred, average="weighted", zero_division=0),  
            f1_score(y_test, boosted_pred, average="weighted", zero_division=0),  
            f1_score(y_test, xgb_pred, average="weighted", zero_division=0),  
            f1_score(y_test, cb_pred, average="weighted", zero_division=0),  
        ],  
        "Precision": [  
            precision_score(y_test, baseline_pred, average="weighted",  
↪zero_division=0),  
            precision_score(y_test, pruned_pred, average="weighted",  
↪zero_division=0),  
            precision_score(y_test, rf_pred, average="weighted",  
↪zero_division=0),  
            precision_score(y_test, boosted_pred, average="weighted",  
↪zero_division=0),  
            precision_score(y_test, xgb_pred, average="weighted",  
↪zero_division=0),  
            precision_score(y_test, cb_pred, average="weighted",  
↪zero_division=0),  
        ],  
        "Recall": [  

```

```

        recall_score(y_test, baseline_pred, average="weighted",
↳zero_division=0),
        recall_score(y_test, pruned_pred, average="weighted",
↳zero_division=0),
        recall_score(y_test, rf_pred, average="weighted", zero_division=0),
        recall_score(y_test, boosted_pred, average="weighted",
↳zero_division=0),
        recall_score(y_test, xgb_pred, average="weighted", zero_division=0),
        recall_score(y_test, cb_pred, average="weighted", zero_division=0),
    ],
}
).set_index("Model").round(4)

print(summary.sort_values("F1 (weighted)", ascending=False).to_string())

```

	Accuracy	F1 (weighted)	Precision	Recall
Model				
XGBoost	0.9474	0.9475	0.9487	0.9474
CatBoost	0.9444	0.9446	0.9451	0.9444
Random Forest	0.9298	0.9301	0.9319	0.9298
Gradient Boosting	0.9298	0.9300	0.9307	0.9298
Baseline Tree	0.9152	0.9153	0.9156	0.9152
Pruned Tree	0.8918	0.8924	0.9011	0.8918

4 Section 4: Feature Importances

```

[ ]: from IPython.display import Markdown, display
display(Markdown("# Feature Importance"))

feature_names = X_train.columns.tolist()

def plot_feature_importance(importances, feature_names, title, top_n=20,
↳color="steelblue"):
    fi_df = (
        pd.DataFrame({"feature": feature_names, "importance": importances})
        .sort_values("importance", ascending=False)
        .head(top_n)
    )
    fig, ax = plt.subplots(figsize=(9, 6))
    ax.barh(fi_df["feature"][::-1], fi_df["importance"][::-1], color=color)
    ax.set_xlabel("Importance")
    ax.set_title(title)
    plt.tight_layout()
    plt.show()
    return fi_df

```

```

# Baseline Decision Tree
plot_feature_importance(
    baseline_tree.feature_importances_,
    feature_names,
    "Baseline Decision Tree - Feature Importance",
    color="slategray",
)

# Pruned Decision Tree
plot_feature_importance(
    pruned_tree.feature_importances_,
    feature_names,
    "Pruned Decision Tree - Feature Importance",
    color="darkorange",
)

# Random Forest
plot_feature_importance(
    rf_best.feature_importances_,
    feature_names,
    "Random Forest - Feature Importance (MDI)",
    color="forestgreen",
)

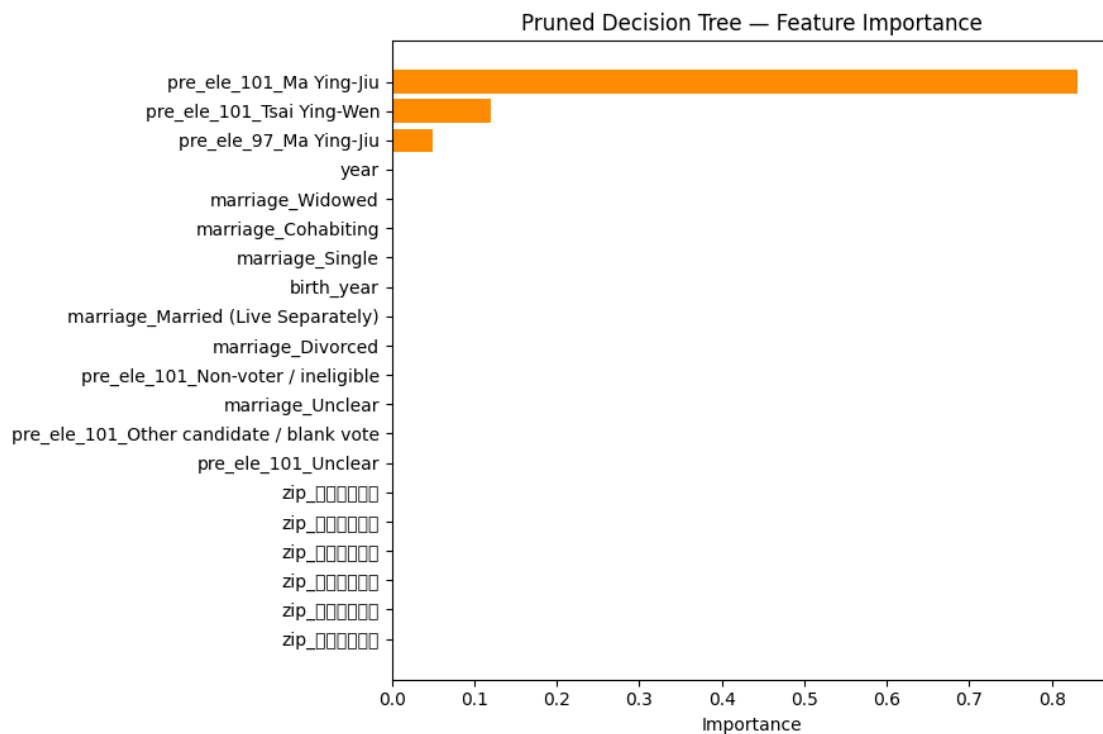
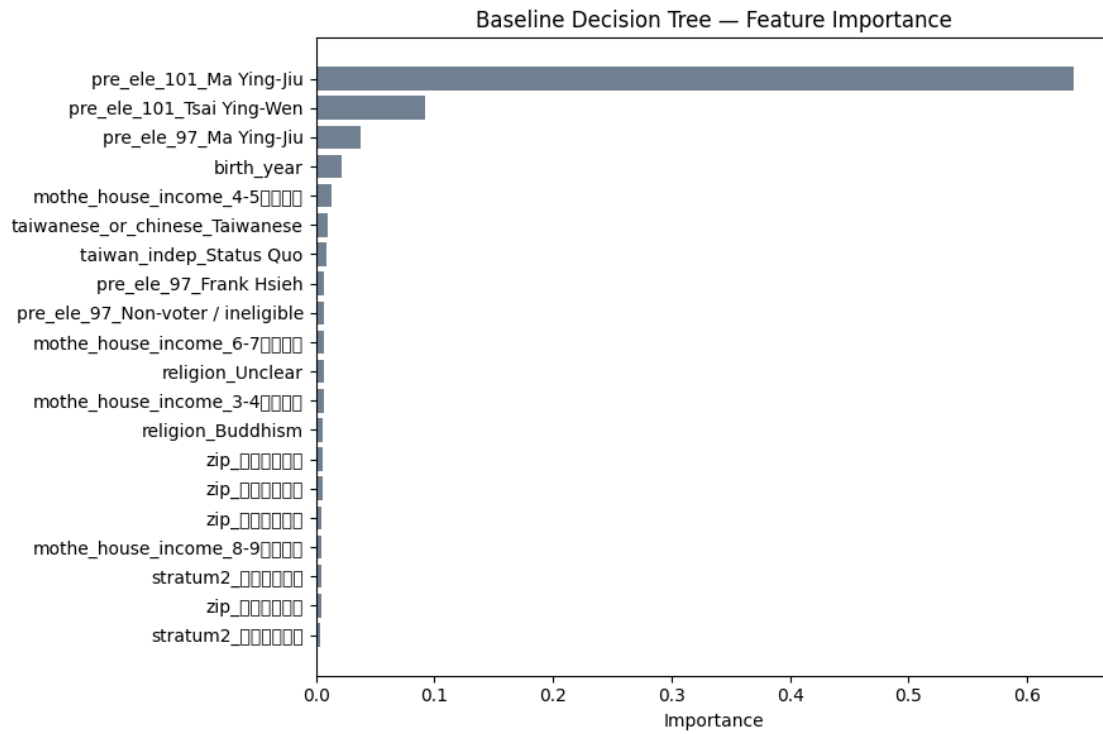
# Gradient Boosting
plot_feature_importance(
    boosted_best.feature_importances_,
    feature_names,
    "Gradient Boosting - Feature Importance",
    color="mediumpurple",
)

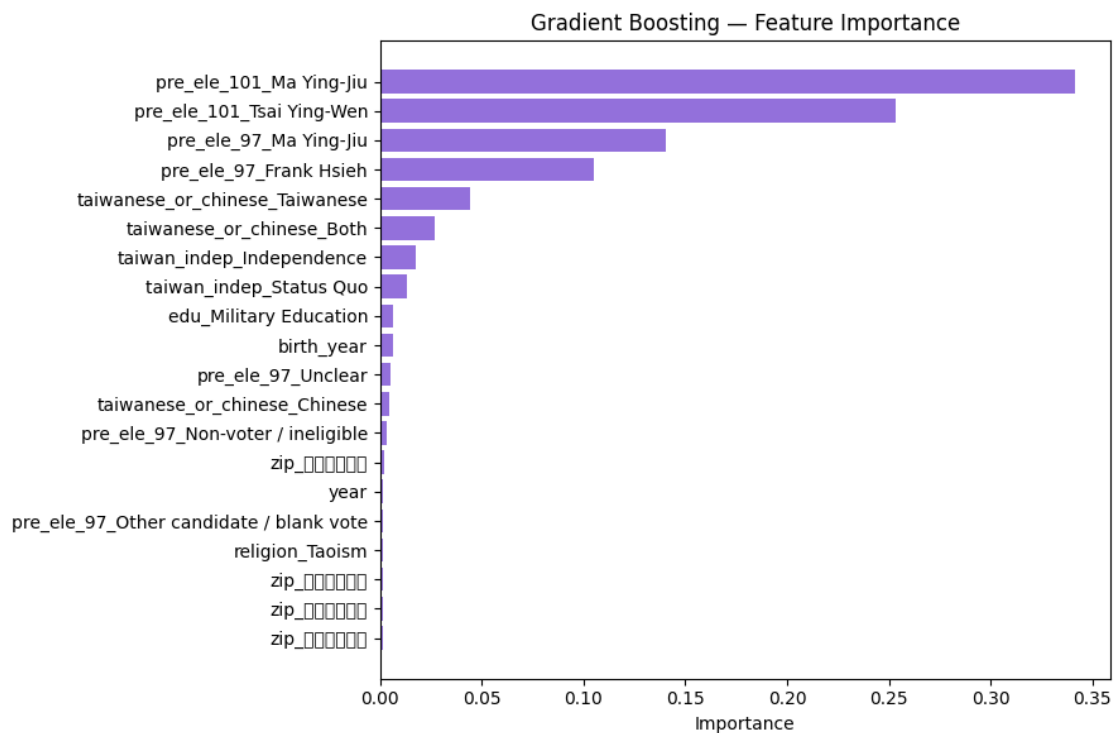
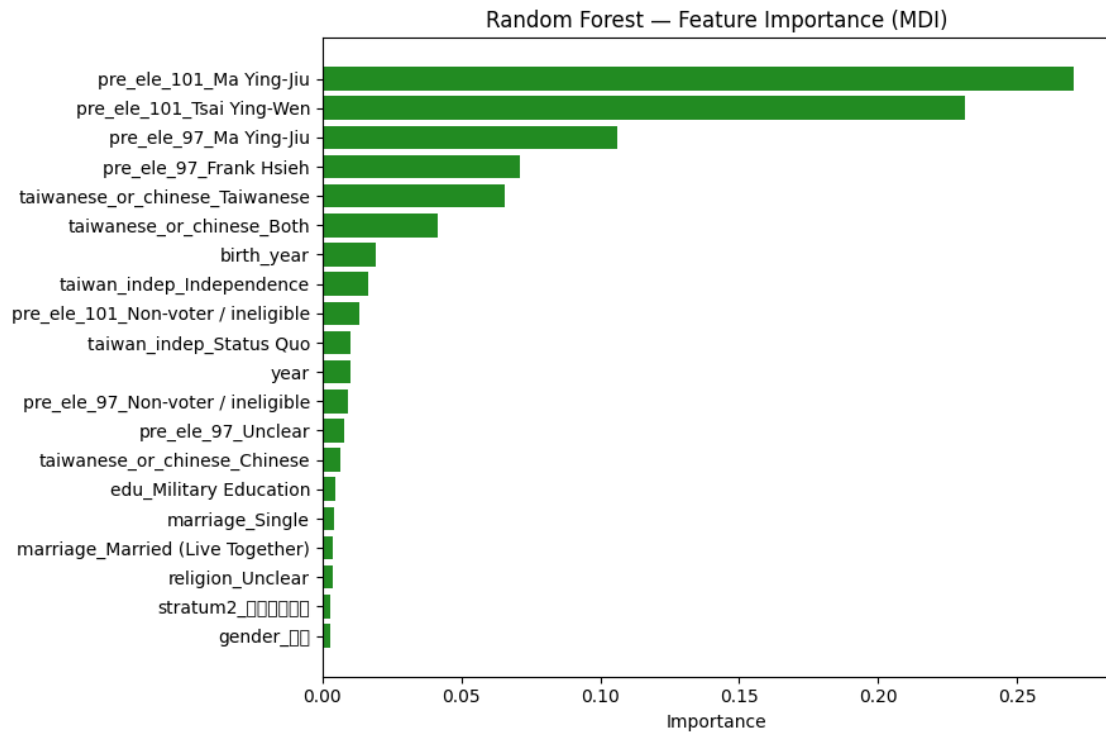
# XGBoost
plot_feature_importance(
    xgb_best.feature_importances_,
    feature_names,
    "XGBoost - Feature Importance (gain)",
    color="dodgerblue",
)

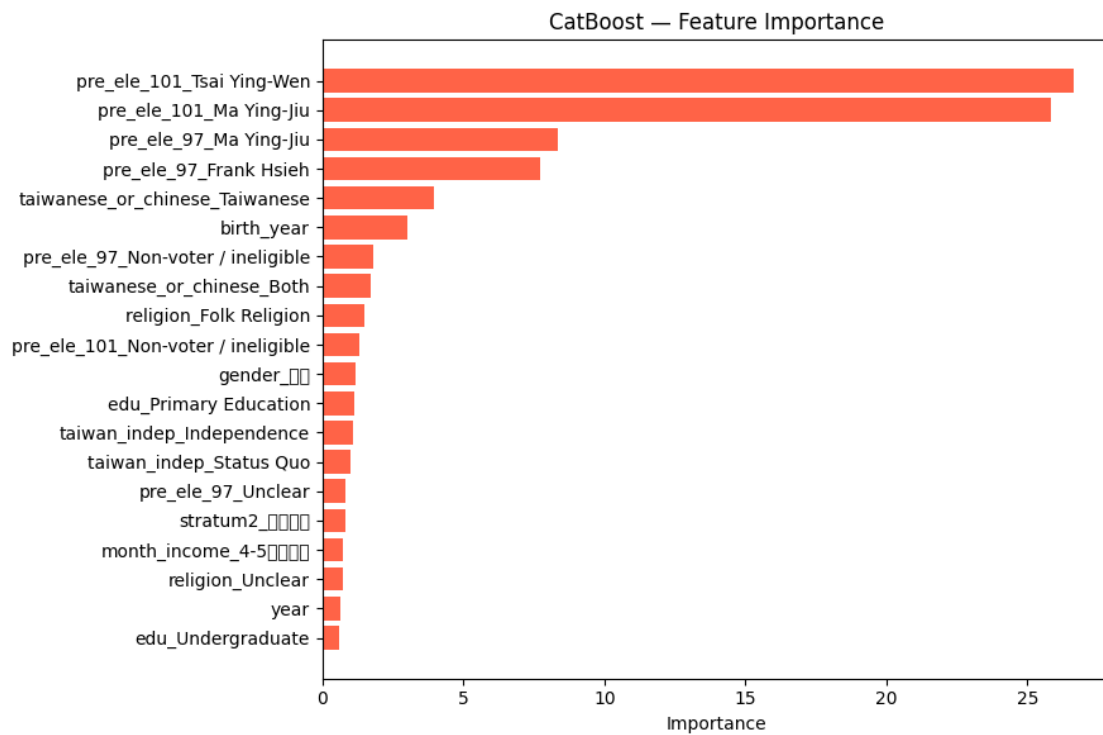
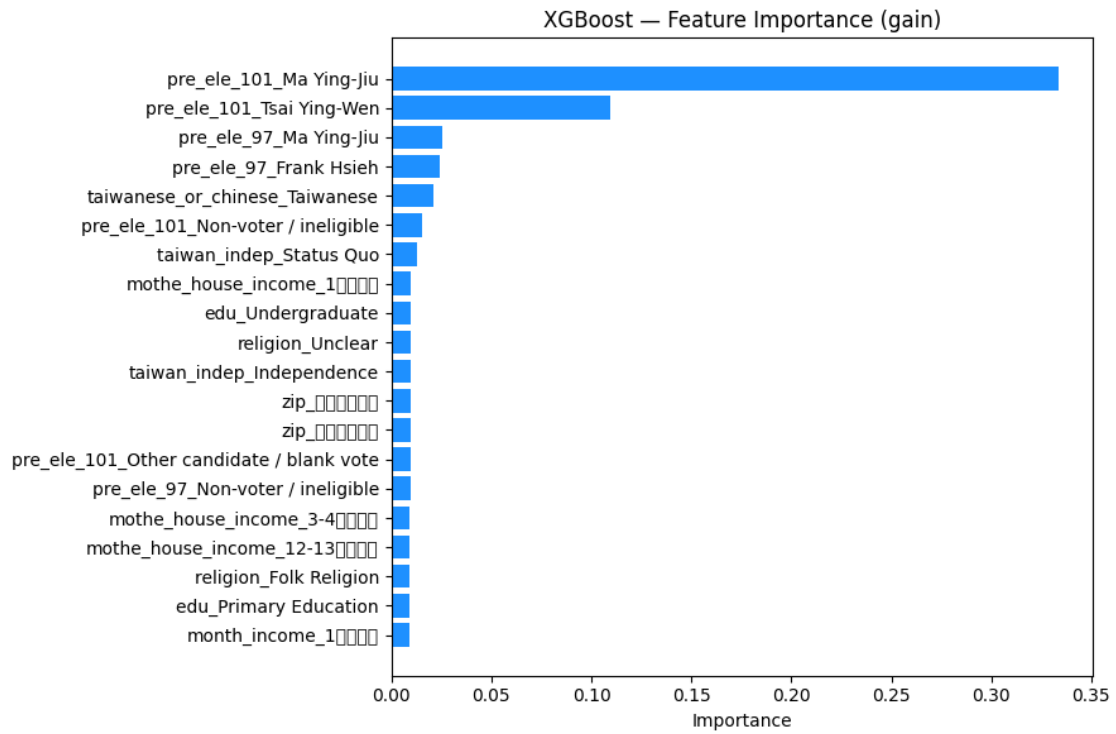
# CatBoost
plot_feature_importance(
    cb_best.feature_importances_,
    feature_names,
    "CatBoost - Feature Importance",
    color="tomato",
)

```

5 Feature Importance







```

[ ]:
      feature importance
10    pre_ele_101_Tsai Ying-Wen 26.657654
11    pre_ele_101_Ma Ying-Jiu  25.818122
164   pre_ele_97_Ma Ying-Jiu   8.356385
163   pre_ele_97_Frank Hsieh   7.713301
147   taiwanese_or_chinese_Taiwanese
1      birth_year              3.005329
162   pre_ele_97_Non-voter / ineligible 1.817301
148   taiwanese_or_chinese_Both 1.700628
140   religion_Folk Religion    1.469655
9     pre_ele_101_Non-voter / ineligible 1.295527
161   gender_                  1.161260
153   edu_Primary Education    1.140521
229   taiwan_indep_Independence 1.072023
228   taiwan_indep_Status Quo  1.014397
165   pre_ele_97_Unclear       0.819653
195   stratum2_                0.800962
205   month_income_4-5         0.744680
145   religion_Unclear         0.735292
0     year                     0.641573
157   edu_Undergraduate        0.568833

```


- 5.0.1 We can see from the above results that the most related predictor of party affiliation is, unsurprisingly, voting history. In this case, voters of Ma Ying-Jiu tend to align themselves more to the KMT, while voters of other candidates and, more strongly, past voters for DPP candidates align themselves with the DPP.
- 5.0.2 As well, it can be emphasized almost how much of an effect ethnic/national self-identification has on party alignment. Here people who regard themselves as Taiwanese, in opposition to Chinese or both Taiwanese and Chinese, tend to be positively correlated with DPP alignment. Whereas, people regarding themselves as Chinese or both Taiwanese and Chinese tend to align themselves more with the KMT Party
- 5.0.3 Same trend seems to happen for stances on Taiwanese independence. DPP supporters seem to tend more likely to support independence, while KMT supporters prefer unification. It is worth to note that supporting Status Quo on Taiwanese independence seems to be more positively correlated with KMT alignment.
- 5.0.4 Most general demographics seem to not be as important to our Random Tree model.
- 5.0.5 We can run a logistic regression model isolating each feature and our target to see how the above important features relate with our target. For example, does `birth_year` relate positively or negatively with our target in each model (i.e Does higher `birth_year` (younger) result in positive or negative correlation to, say, KMT?)

```
[ ]: isolated_features = [
    "birth_year",
    "taiwanese_or_chinese",
    "pre_ele_101",
    "pre_ele_97",
    "taiwan_indep",
]

X_isolated_features = pd.DataFrame(X, columns=isolated_features)

X_isolated_features = pd.get_dummies(
    X_isolated_features,
    columns=X_isolated_features.select_dtypes(include=["category"]).columns,
)

y_by_target = pd.get_dummies(y).astype("int")
```

```
[ ]: from sklearn.linear_model import LogisticRegression
from scipy.stats import pearsonr

positive_coefficients = []
```

```

negative_coefficients = []

for column_y in y_by_target.columns:
    if column_y != "Other":
        for column_X in X_isolated_features.columns:

            current_feature = X_isolated_features[column_X]
            current_target = y_by_target[column_y]

            # Build a simple linear regression model
            model = LogisticRegression()
            model.fit(current_feature.to_frame(), current_target)

            # Print the intercept and coefficient
            # print(f"Intercept: {model.intercept_}")
            # print(f"Coefficient for {column_X} on {column_y}: {model.
↪coef_[0]} \n")

            if model.coef_[0] > 0.90:
                positive_coefficients.append([column_X, model.coef_[0]])
            elif model.coef_[0] < -0.90:
                negative_coefficients.append([column_X, model.coef_[0]])

            # Print features with positive and negative coefficients
            print(
                f"\n The features with positive correlation with our target_
↪{column_y} are:"
            )
            for coef in positive_coefficients:
                print(f"{coef[0]} with coefficient {coef[1]}")

            print(
                f"\n The features with negative correlation with our target_
↪{column_y} are:"
            )
            for coef in negative_coefficients:
                print(f"{coef[0]} with coefficient {coef[1]}")

positive_coefficients = []
negative_coefficients = []

```

The features with positive correlation with our target DPP are:
taiwanese_or_chinese_Taiwanese with coefficient [2.14237866]
pre_ele_101_Tsai Ying-Wen with coefficient [4.95874806]
pre_ele_97_Frank Hsieh with coefficient [4.12821962]
pre_ele_97_Other candidate / blank vote with coefficient [1.00546181]

taiwan_indep_Independence with coefficient [2.01266087]

The features with negative correlation with our target DPP are:

taiwanese_or_chinese_Both with coefficient [-1.8898267]
taiwanese_or_chinese_Unclear with coefficient [-1.22438327]
taiwanese_or_chinese_Chinese with coefficient [-2.00574056]
pre_ele_101_Ma Ying-Jiu with coefficient [-4.82312458]
pre_ele_97_Ma Ying-Jiu with coefficient [-3.27012106]
taiwan_indep_Status Quo with coefficient [-1.005583]
taiwan_indep_Unification with coefficient [-0.9783651]

The features with positive correlation with our target KMT are:

taiwanese_or_chinese_Both with coefficient [1.8898267]
taiwanese_or_chinese_Unclear with coefficient [1.22438327]
taiwanese_or_chinese_Chinese with coefficient [2.00574056]
pre_ele_101_Ma Ying-Jiu with coefficient [4.82312458]
pre_ele_97_Ma Ying-Jiu with coefficient [3.27012106]
taiwan_indep_Status Quo with coefficient [1.005583]
taiwan_indep_Unification with coefficient [0.9783651]

The features with negative correlation with our target KMT are:

taiwanese_or_chinese_Taiwanese with coefficient [-2.14237866]
pre_ele_101_Tsai Ying-Wen with coefficient [-4.95874806]
pre_ele_97_Frank Hsieh with coefficient [-4.12821962]
pre_ele_97_Other candidate / blank vote with coefficient [-1.00546181]
taiwan_indep_Independence with coefficient [-2.01266087]

6 Final Insights from Machine Learning Models

Based on the comprehensive modeling and correlation analysis above, we can draw the following key conclusions:

1. **Model Performance Hierarchy:** Advanced gradient boosting frameworks (XGBoost and CatBoost) outperformed the standard Random Forest and simple Decision Trees. XGBoost achieved the highest accuracy and F1 score (~94.7%), showing that iterative error correction on this dataset yields highly accurate partisan predictions.
2. **Dominance of Past Voting Behavior:** Across all tree-based models, `pre_ele_101` and `pre_ele_97` (past presidential voting choices) consistently ranked as the most important features by a massive margin. Partisan loyalty is extremely sticky; how an individual voted in previous elections is the strongest predictor of their baseline party affiliation.
3. **Ideological and Identity Polarization:** The logistic regression coefficients perfectly map the core ideological divides between the two major parties.
 - **DPP Alignment** is highly correlated with self-identifying exclusively as “Taiwanese” and supporting Taiwanese “Independence”.
 - **KMT Alignment** is highly correlated with identifying as “Both” (Taiwanese and Chinese) or “Chinese”, and supporting either the “Status Quo” or “Unification”.
4. **Demographics vs. Ideology:** General demographics like birth year, marriage status, and income showed surprisingly little weight compared to identity and historical voting patterns,

implying that Taiwanese political affiliation is driven much more by national identity issues than standard socioeconomic dividing lines.